PRODUCT ENGINEERING / VERSION 1.0

AR + AI Smart Placement System

A browser-first spatial furniture placement and fit-validation platform

Technical research, current-prototype review, functional requirements, architecture, implementation phases, test cases, and delivery criteria.

Prepared for: Product owner and development team | Primary platform: Android Chrome WebXR | Research date: 02 November 2026

Product principle Let users choose a location, show a true-scale product footprint, scan nearby geometry, and validate fit before displaying the final 3D product. Unknown space must never be interpreted as free space.

Decision summary

- Keep the existing product selection, user-directed target, move/rotate controls, and informative placement result UI.
- Replace purely visual green/red segmentation as a fit oracle with metric spatial geometry + occupancy + deterministic collision/clearance tests.
- Introduce preliminary true-scale footprint BEFORE target confirmation, then guided multi-view scanning and an explicit UNCERTAIN status.
- Do not assume iOS Safari has feature parity with Android immersive WebXR. Offer a lower-confidence fallback or optional native precision path.
- Raw video retention or automatic gallery saving is not required for placement validation; recording/export is a separate, consent-driven feature.

1. Scope, intent and evidence

1.1 Desired user journey

The user selects a furniture product, enters the camera, previews a real-size target outline on the floor, confirms the chosen position, scans nearby space from multiple viewpoints, receives a transparent fit judgment, and can then move or rotate the anchored 3D product while validation updates. The user—not AI—owns the final position choice.

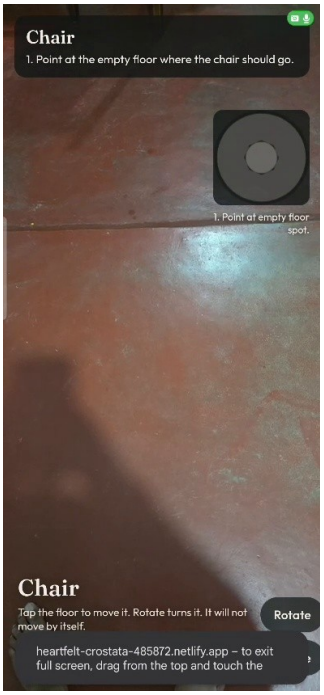
1.2 What the supplied 58-second video actually shows

Observed directly: a mobile web product card; a Chair product labeled 77 × 85 × 86 cm; a prominent circular target; guidance to point at floor and turn; green/red overlays over parts of the room and furniture; “Fits here” and “Does not fit” messages; controls including Rotate, Nearby spot, Accept, and Ask. The validation outcome changes as the view moves.

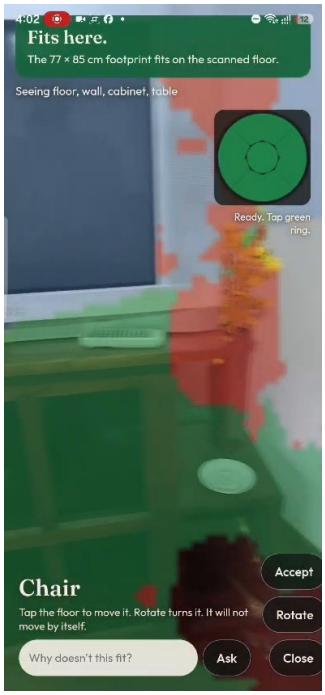
Not verifiable from video alone: whether WebXR immersive-ar is active, which features are enabled, whether raw camera frames or metric depth are accessed, the segmentation model architecture, the

coordinate calibration, and how real-world collisions are computed. Source inspection and device logging are required before retaining or replacing these internals.

1.3 Prototype evidence frames (user-provided video)



5 s — target and guidance



13 s — fit status / overlay



24 s — invalid status on chair

Interpretation: these stills show user-facing UI, not proof of metric geometry quality. The visible masking on furniture warrants a dedicated floor-vs-obstacle test.

2. User experience specification: eight steps

Step	UI and interaction	Engine actions and exit condition
01. Open camera	Product page → View in room → user permission; loading and capability status.	Probe immersive-ar; request session on tap, enable hit-test, local-floor, anchors, optional planes/depth/camera; choose supported mode.
02. Display floor target	Small locator ring + product-sized translucent rectangular/polygon footprint with dimensions; “Searching for floor…” until confident.	Filter hit test to viable floor candidate; floor classifier checks semantics + height/normal + area + temporal stability.
03. Confirm target	User drags/aims, rotates footprint, taps “Confirm location”.	Save fixed floor-relative transform (anchor or tracked pose); store selectedYaw and requested footprint.
04. Guided scan	Ask user to view target from several sides; progress by coverage, not arbitrary 360° timer.	Fuse metric depth samples and XR poses into target-local occupancy/height map; update boundaries and tracking confidence.
05. Measure fit	Show real dimensions, occupied zones, observed floor boundaries, and unknown zones.	Run footprint-within-floor, overlap, 3D obstacles, wall intersections, and required/recommended clearances.

06. Explain invalid	Show blocked side and whether place is too narrow, obstructed, or insufficiently scanned; 3 user options.	Use deterministic reason codes; optional computer-vision label to describe blocker; never claim exact obstacle class without confidence.
07. Place 3D model	On VALID / acknowledged WARNING, reveal anchored true-size GLB, shadow and occlusion.	Retain pose/world scale; lighting estimate if available; perform independent visual and collision model transforms.
08. Edit and re-check	Move/rotate using controlled gestures; revalidate continuously or on release; reset/scan again.	Any pose change invalidates previous fit certificate. Recompute result and display before “confirmed” state.

3. Critical UX adjustment: product-sized footprint before scanning

Strongly recommended. A plain circle identifies a point but does not communicate a chair/sofa’s occupied region. Use two synchronized visuals: (a) thin central locator ring/crosshair, and (b) actual product shape on the floor, using width, depth and yaw from product metadata.

Chair example (video): width = 0.77 m, depth = 0.85 m, height = 0.86 m

Initial floor outline: 0.77 m × 0.85 m (true geometric scale)

With optional 0.10 m side clearance: recommended overall width = 0.97 m

Show body footprint separately from access/comfort clearance (dashed outline).

- Preview states: gray = floor candidate not ready; teal/neutral = selectable, not yet validated; green = validated; amber = limited clearance; red = blocked; gray-striped = unknown / rescan.
- Do not imply fit before scanning. The selected target is provisional and the result is a separate property.
- If camera poses shift (tracking relocalization / anchor drift), update/reacquire the target and invalidate stale measurements.

4. Scan experience: improve the requested “360-degree turn”

The proposed guided 360° turn is useful as a coaching concept, but rotating in place does not observe behind a sofa, under cabinetry, or other occluded regions. The user should stand next to—not on—the target footprint and slowly sweep the phone around the floor, nearby objects, wall edges and the open walking path, adding one or two small lateral viewpoint moves when safe.

Safety and validity Do not instruct the user to step into a tight or obstructed area. Show a “move to the side and look back” option. Require scan coverage of the actual candidate volume and clearance zone, not just a 360° angle counter.

Completion criteria are configurable and should be tuned on recorded test sets: percentage of candidate footprint observed, boundary confidence, target-height observations, pose stability and revisit agreement. Missing data yields UNCERTAIN and targeted rescan prompts, never VALID.

5. Core sensing design: floors, walls, obstacles

5.1 Floor is not the same as “any horizontal surface”

A chair seat or table top often appears horizontal. Use a multi-signal floor confidence score over XRPlane candidates: semanticLabel when present; up-facing normal; clustering by world height; large/persistent planar region; overlap with depth near expected floor; adjoining vertical boundaries. No single signal

should decide. Semantic labels may be null; plane detection should be optional if not available on the runtime.

5.2 Spatial model / occupancy grid

MVP should use a 2.5D target-local map (e.g. 2–5 cm per cell for initial experiments, tune performance). Each cell carries occupancy/unknown state, lowest/highest observed height where useful, observation count, age, depth confidence, and estimated dynamic/static flag. Do not collapse “never observed” to empty floor.

XR pose + calibrated metric depth → back-project valid pixels to XR world
→ transform into a stable floor/target coordinate system
→ remove floor inliers (within sensor-noise threshold)
→ fuse above-floor points into occupied geometry
→ maintain UNKNOWN coverage and temporal evidence
→ run collision/clearance against product collision proxies.

5.3 Boundary conditions

- Wall: XR vertical plane and polygon when available; corroborate with depth and segmentation. A large detected wall plane does not necessarily guarantee the full wall is visible.
- Floor: usable polygon minus holes, thresholds, steps and immovable obstacles. Segmentation masks are semantic hints—not physical dimensions.
- Obstacles: chairs, cabinets, protruding shelves, table edges, doors and hanging items must be checked in 3D, not floor pixels only.
- Occlusion: surfaces hidden from every camera pose remain UNKNOWN. An empty-looking 2D image must not imply empty 3D space.

6. Where WebXR and AI segmentation enter

Component	Purpose	Trust level / fallback
WebXR session + hit test	Real-world coordinate frame and floor targeting.	Primary when supported. A normal getUserMedia feed alone does not provide metric XR coordinates.
WebXR plane detection	Horizontal/vertical candidate polygons and optional semantic labels.	Optional; check enabledFeatures and existence on each device. Some labels are absent.
WebXR depth sensing	Depth per XR view; supports geometry and occlusion.	Primary metric source, with quality checks; CPU/GPU access depends on negotiated mode.
WebXR anchors	Track selected furniture origin over camera movement.	Use if granted, else maintain pose and track drift explicitly.
Camera-access feature	Access XR-aligned camera texture for on-device semantic segmentation.	Only if browser/runtime grants raw-camera-access. Separate camera-stream mode cannot be assumed pose-aligned.
Segmentation model	Classify floor/wall/furniture pixels; enhance prompts and exclude semantic false positives.	Supporting evidence only; no direct measuring/fit decision.
Placement validator	Compute actual fit from observed geometry, product dimensions and configurable rules.	Authoritative outcome; explain via reason codes.

Optional text AI	Explain deterministic result conversationally; suggest next action.	Must not invent measurements or override UNKNOWN.

6.1 XR-aligned AI segmentation data flow

XR frame and view (same timestamp)
↘ Metric depth + transform ↙ XR camera image (only if camera-access granted)
world-space geometry 2D semantic mask (floor / wall / furniture)
↘ ↙
registered evidence fusion
↓
occupancy + semantic tags + confidence
↓
deterministic fit engine → result / explanation

Important implementation detail: if the chosen browser does not provide XR-aligned camera pixels, do not silently mix getUserMedia frames with XR poses. A separate pre-scan mode can run image segmentation, but cross-calibration, timing, and world-scale alignment are an explicit research task; segmentation should remain advisory when calibration is unreliable.

6.2 Segmentation implementation choices

- Research path: small on-device segmentation model (for example SegFormer / DeepLab family) served through ONNX Runtime Web or TensorFlow.js; choose based on real device latency, thermal load, footprint, privacy and output quality.
- Prefer worker-based inference and throttled sampling; avoid running segmentation on every high-frame-rate XR render callback.
- Mask confidence and temporal consistency should be logged. CV “chair” recognition may improve user wording but geometry determines collision.
- If raw-camera feature, CPU depth or required permissions are unavailable, explicitly enter a degraded mode rather than claiming measured accuracy.

7. Deterministic placement rules

7.1 Minimum product data contract

```
ProductPlacementAsset {
 productId: string; glbUrl: string; // USDZ optional for Quick Look
 widthM: number; depthM: number; heightM: number;
 localOrigin: 'floor-center'; // world-up Y, front orientation documented
 collisionShapes: OBB[] | ConvexProxy[];
 mounting: 'floor' | 'wall' | 'tabletop';
 requiredClearanceM: {front:number; back:number; left:number; right:number};
 recommendedClearanceM: {front:number; back:number; left:number; right:number};
 variants?: string[]; assetVersion: string;
}
```

7.2 Validation checks (deterministic)

1. Target support: requested floor candidate is stable; actual chair legs / support footprint rests on an observed, supportable plane.

2. Footprint: rotated product body footprint remains inside observed usable polygon with configured tolerance.
3. Solid intersection: 3D product collision proxy does not intersect sampled obstacle volume, room boundaries or protected fixtures.
4. Clearance: essential safety/function clearances produce INVALID if violated; recommended comfort margins can produce WARNING.
5. Height/overhang: consider shelves, table tops and wall-hung objects intersecting the real product volume.
6. Unknown/poor confidence: if critical voxels/cells are unobserved, or pose drift/metric accuracy is too low, return UNCERTAIN.
7. Re-check after every move/rotate, product variant/size change, new obstacle data or tracking reset.

7.3 Fit outcome

Status	Meaning	What user sees
VALID	All required physical checks clear and scan confidence sufficient.	“It fits here.” Show product.
WARNING	Physical body fits, but optional/recommended clearance is small.	“It fits, but walking space is tight.” User acknowledges.
INVALID	Measured collision or violation of a required constraint.	“Needs 24 cm more width” / blocked side / next actions.
UNCERTAIN	Insufficient scan, unsupported metric source, or tracking instability.	“Scan this corner again” — do not issue fit claim.

A measured deficit must be computed from geometry and reported with uncertainty. Example 24 cm below is illustrative; it is not a measurement of the user’s supplied video.

```
{
 "status": "INVALID",
 "reasonCodes": ["INSUFFICIENT_WIDTH", "OBSTACLE_COLLISION"],
 "productId": "chair-001", "sceneVersion": 16, "poseVersion": 8,
 "requiredWidthM": 0.97, "observedAvailableWidthM": 0.73,
 "estimatedDeficitM": 0.24, "confidence": 0.91,
 "obstacleSide": "right", "needsRescan": false
}
```

8. Recommended application architecture

```
Commerce/Product UI (React / Next.js / TypeScript)
   |
Capability Router — supported features / fallback selection
   |
XR Capture Adapter (Three.js + WebXR, XR poses, hit test, planes, depth, anchors)
   + Optional XR-aligned camera-access → segmentation worker
   |
Spatial Fusion Service (floor model / wall candidates / target-local occupancy)
   |
Placement Engine (OBB + swept clearance + confidence and reason codes)
   |                    |
```

AR Visual Layer / UI Explanation / Next-step text

|

Telemetry (metrics only), consentful persistence, non-sensitive backend

8.1 Suggested TypeScript modules

src/features/ar/

capability/capabilityRouter.ts

xr/xrSessionManager.ts

xr/hitTestController.ts

xr/planeTracker.ts

xr/depthAdapter.ts

xr/anchorManager.ts

vision/segmentationWorker.ts

vision/frameAlignment.ts

scene/floorClassifier.ts

scene/spatialGrid.ts

scene/worldFusion.ts

placement/footprintPreview.ts

placement/collisionEngine.ts

placement/clearanceRules.ts

placement/fitEvaluator.ts

ui/scanCoach.tsx

ui/placementControls.tsx

diagnostics/qualityMetrics.ts

8.2 Minimal state machine

IDLE → CAPABILITY_CHECK → AWAIT_PERMISSION → FLOOR_SEARCH

→ TARGET_PREVIEW → TARGET_CONFIRMED → SCANNING

→ VALIDATING → {VALID | WARNING | INVALID | UNCERTAIN}

→ {PLACED | TARGET_PREVIEW | SCANNING}

→ EDITING → VALIDATING → ...

Always handle: SESSION_LOST, TRACKING_LOST, PERMISSION_DENIED, DEVICE_UNSUPPORTED.

9. Capability matrix and platform choices

Platform / mode	What is realistic	Product promise
Android Chrome + supported ARCore depth	Full browser-first WebXR fit-validation prototype; exact feature set must be detected at runtime.	Measured fit when geometry coverage + confidence pass.
Android Chrome without usable depth	Hit-test / visualization, perhaps plane-based coarse fit; cannot reliably detect furniture geometry.	Preview only / uncertain / manual tape measurement option.
iPhone Safari standard browser	Immersive WebXR unavailable; native Quick Look/USDZ can preview model but not run full custom in-session geometry validator.	AR preview, not equivalent measured fit validation.

iPhone/iPad with LiDAR + optional native helper	RoomPlan provides more structured geometry through Swift/ARKit if app component allowed.	Higher-quality scan after separate integration / device testing.
Desktop / unsupported browser	Interactive 3D viewer and product dimensions.	No live physical fit claim.

WebXR Plane Detection reached Chrome 147 stable on 7 April 2026. API support is not equivalent to reliable availability on all Android devices. Any needed WebXR feature should be feature-detected, optional vs required selected deliberately, and tested on actual Android devices. iOS Safari WebXR non-support is acknowledged by WebKit as of March 2026.

10. User video / photo-gallery policy

The project brief mentions screen video / gallery auto-save. This needs a separate user-product decision. On the web, unattended automatic saving to a user's Photos/Gallery is not a dependable browser capability. Screen recording via getDisplayMedia is permissions-gated, inconsistent on mobile, and not necessarily able to capture XR compositor passthrough frames.

- Recommendation for V1: DO NOT continuously record or auto-upload the room. Store minimal, transient spatial geometry locally in session. No screen-video auto-save.
- If users explicitly request recording: add a separate opt-in "Record / Share" feature, show browser permission, export a compatible media file through user action, and test actual device behavior. Never promise silent gallery saving.
- If automatic gallery saving is an absolute business requirement, assess a native companion component; do not implement by relying on browser workarounds.

11. Performance, privacy, accessibility and QA

11.1 Performance implementation targets (initial engineering targets, not promises)

- Keep AR render responsive; aim for ≥ 24 FPS on test reference devices, with dynamic quality adaptation. Measure p50/p95 FPS and thermal throttling.
- Throttle segmentation; prefer 2–10 inference frames per second as an experiment, rather than assuming it can sustain every render frame.
- Keep GLB low-poly, correct scale, with meshopt/Draco where helpful, KTX2 mobile-friendly textures, explicit LOD and cached delivery.
- Occupancy grid cell size, voxel height bands, depth temporal window, and maximum working set must be measured and tuned by device.

11.2 Privacy

- Camera and spatial environment are sensitive: use HTTPS, least-privilege permissions, clear indicator, and session-local default processing.
- Do not send raw room images to LLMs by default. Provide reason codes, observed dimensions and confidence; redact or avoid personal room metadata.
- Provide clear user choice for persistence, delete/cancel scan, and explicit opt-in for any telemetry with spatial content.

11.3 Accessibility

- Avoid color-only feedback. Provide icons, visible text and haptic/voice cues where available; clear Undo/Reset and motion-safe scan guidance.

12. Test plan and acceptance criteria

Test on multiple named Android devices, chipsets, lighting conditions and Chrome versions. Retain ground-truth tape/laser measurements with consent. Compare measured clearances to physical truth. A single attractive demo is not sufficient for “fits” claims.

ID	Test scenario	Expected result
T01	Point at an empty floor next to a chair seat	Chair seat must not be designated as floor.
T02	Preview 77 × 85 cm chair footprint	Width/depth displayed at correct scale/orientation, independently verified.
T03	Fit target clearly inside observed open floor	VALID if metric measurements and coverage meet confidence rules.
T04	Chair target overlaps real chair or table leg	INVALID and correct blocked zone highlighted.
T05	Target overlaps unscanned area behind furniture	UNCERTAIN, targeted scan prompt; never VALID.
T06	Scan by stationary 360° spin only	Unknown regions must remain unknown; scan not automatically “complete”.
T07	Change product yaw by 90° next to wall	Footprint/collision and state recomputed.
T08	Temporarily lose tracking or anchor	Pause / UNCERTAIN; re-acquire; no stale green result.
T09	Bright window, mirror, dark carpet, blank wall	Quality warning or reduced confidence when perception fails.
T10	No depth / no camera-access / no planes	Graceful degraded capability mode; no false measured-fit claim.
T11	iPhone Safari	Fallback clearly labeled; no pretending immersive WebXR fit is available.
T12	Recording / save feature if shipped	Prompts + explicit user gesture; zero silent camera uploads.

12.1 Release gates

- G1: Pose and metric-scale correctness demonstrated on ground-truth target positions in three varied rooms.
- G2: No false-VALID results in the agreed high-risk obstacle/unknown-area regression suite; all false negatives and uncertainties logged.
- G3: Device/browser support matrix based on real runtime tests, not browser version assumption alone.
- G4: Tracking reset, scan partial coverage, segmentation misclassification, privacy consent and no-device-support flows pass QA.

Suggested research KPI candidates (to validate, not hard-coded promises): placement pose drift and dimension error distributions at 0.5–3 m; observed-region percentage; collision precision/recall; false-VALID

rate and UNCERTAIN rate; latency from scan complete to result; 95th-percentile frame time. No numeric “guaranteed centimeter accuracy” should be marketed until measured.

13. Proposed development milestones

Milestone	Implementation focus	Demo / evidence required
M0. Audit existing prototype	Review repository, dependencies, XR feature support and real source of mask/green-red fit status.	Architecture audit + retained/replaced module list.
M1. Metric target preview	Android WebXR session, correct floor classification, product-size rectangle and anchored location.	Chair footprint at real tape-measured size; no table/seat floor false positives.
M2. Scan + spatial model	Depth ingestion, multi-view fusion, occupancy/unknown zones, basic UI scan coaching.	Obstacle grid and coverage visualizer on real device.
M3. Trusted fit validator	OBB, floor polygon, wall/obstacle collision, clearances, 4 outcomes.	Ground-truth fit/not-fit/uncertain test matrix.
M4. Production AR editing	GLB rendering, anchored move/rotate, dynamic revalidation, elegant result UX.	Stable e-commerce integrated end-to-end flow.
M5. AI CV + language (optional)	XR-aligned segmentation where possible; meaningful obstacle labels; structured natural-language explanations.	Ablation test: does AI improve classification without degrading fit truth?
M6. Cross-platform + hardening	iOS fallback/native feasibility, perf, privacy, analytics, device coverage.	Supported-device documentation + release readiness review.

14. Decisions the developer must make with product owner

8. Is the product promise “visualize approximately” or “measured physical fit”? This specification assumes measured fit on a qualified platform, with visible uncertainty.
9. Does a tap-to-view 3D preview on iOS suffice for V1, or is a native iOS LiDAR/RoomPlan scan essential?
10. Product catalog and clearance policy: required physical safety clearance vs recommended walking/comfort clearance for each product category.
11. Are screenshots/videos optional user-requested exports, or is a native gallery integration a hard requirement?
12. Will temporary room geometry ever be saved or linked to user accounts? If so, obtain explicit privacy approval and retention policy.
13. What exact model/device versions form the Android release support matrix?

15. References and technology validation

Sources checked against official release notes, standards, vendor documentation and browser platform references. They support API availability and limitations, not proof of accuracy for this particular prototype.

R1 Chrome 147 release notes — WebXR Plane Detection (stable 7 Apr 2026)
https://developer.chrome.com/release-notes/147

R2 W3C WebXR Depth Sensing Module (25 Aug 2026 Working Draft)
https://www.w3.org/TR/webxr-depth-sensing-1/

R3 WebXR Plane Detection specification — semantic labels and polygon
https://immersive-web.github.io/plane-detection/

R4 WebXR Raw Camera Access Module — camera-access and frame-aligned texture
https://immersive-web.github.io/raw-camera-access/

R5 Google: WebXR compared with ARCore — supported runtime features
https://developers.google.com/ar/develop/webxr/arcore-comparison

R6 Google: Depth adds realism — depth from motion and measurement limitations
https://developers.google.com/ar/develop/depth

R7 Apple: RoomPlan — native Swift and LiDAR room scanning
https://developer.apple.com/augmented-reality/roomplan/

R8 WebKit bug 309550 — explicit iOS WebXR support limitation (Mar 2026)
https://bugs.webkit.org/show_bug.cgi?id=309550

R9 MDN XRFrame.getDepthInformation — runtime limitations and API contract
https://developer.mozilla.org/en-US/docs/Web/API/XRFrame/getDepthInformation

R10 MDN getDisplayMedia — permissions and limited browser support
https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getDisplayMedia

R11 8th Wall migration FAQ — status of old hosted platform
https://8thwall.org/docs/migration/faq

R12 WebXR Hit Test Module — native real-world hit-test inputs
https://www.w3.org/TR/webxr-hit-test-1/

Appendix A. Developer validation checklist

- ☐ Prove that the AR origin and GLB local origin are in consistent meters and up-axis.
- ☐ Log granted WebXR features and depth usage per session, and handle null frames.
- ☐ Ensure floor semantic classifier can reject a chair seat and a coffee-table surface.
- ☐ Use target-local pose stabilization and a scene version ID for measurements.
- ☐ Keep UNKNOWN as a first-class grid state and propagate it into fit evaluator.
- ☐ Segmentation source must be timestamp-aligned with XR view before projected fusion.
- ☐ Do not treat text/LLM response as authority over geometry/validation state.
- ☐ Use separate visual mesh and simple collision proxies for each product variant.
- ☐ Validate every move and rotation; block stale fit results after tracking loss.
- ☐ Clearly disclose supported Android model/device browsers and iOS fallback behavior.
- ☐ No automatic room video saving / upload; recording is an opt-in separate feature.
- ☐ Report dimension accuracy and false-VALID rates from measured physical test scenarios.